endava

UNIVERSITATEA DIN BUCUREȘTI
VIRTUTE ET SAPIENTIA

PROGRAMARE ORIENTATA PE OBIECTE II

CURS 6

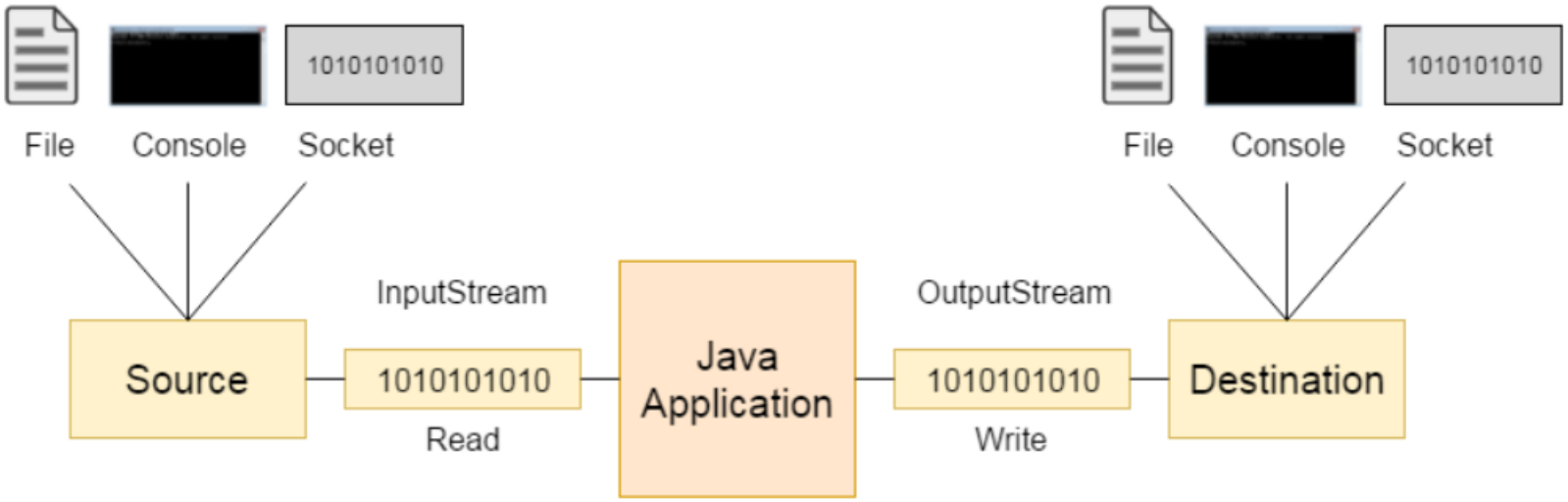
AGENDA CURS 5

1. I/O streams
2. Byte streams
3. Character streams
4. Try-with-resources
5. NI/O 2 streams

I/O STREAMS

- **I/O Streams** sunt folosite in Java pentru a citi si a scrie date din/spre surse externe: fisiere, retea, memoria, etc.
- Sunt implementate cu clase din pachetul **java.io**
- In functie de directia in care circula informatia, relativ la aplicatia Java, sunt impartite in doua categorii:
  - **Input streams**
  - **Output streams**
- In functie de tipul de informatie care circula, sunt impartite in doua categorii:
  - **Byte Streams** (pentru date binare)
  - **Character Streams** (pentru text)

BYTE STREAMS



endava

BYTE STREAMS

1. Fisier

Tip	Clasa	Descriere
Citire	`FileInputStream`	Citeste octeti dintr-un fisier
Scriere	`FileOutputStream`	Scrie octeti intr-un fisier

---

2. Memorie (array de byte)

Tip	Clasa	Descriere
Citire	`ByteArrayInputStream`	Citeste dintr-un array de octeti
Scriere	`ByteArrayOutputStream`	Scrie intr-un array de octeti (in memorie)

endava

BYTE STREAMS

- 3. **Buffered (performanta mai buna)**

Tip	Clasa	Descriere
Citire	`BufferedInputStream`	Citire cu buffer pentru viteza
Scriere	`BufferedOutputStream`	Scriere cu buffer pentru viteza

- 4. **Date primitive (int, double, boolean, etc.)**

Tip	Clasa	Descriere
Citire	`DataInputStream`	Citeste tipuri primitive in format binar
Scriere	`DataOutputStream`	Scrie tipuri primitive in format binar

endava

BYTE STREAMS

- 5. **Obiecte serializate**

Tip	Clasa	Descriere
Citire	`ObjectInputStream`	Citeste obiecte serializate dintr-un stream
Scriere	`ObjectOutputStream`	Scrie obiecte serializate

endava

CHARACTER STREAMS

1. Fisier

Tip	Clasa	Descriere
Citire	`FileReader`	Citeste caractere dintr-un fisier
Scriere	`FileWriter`	Scrie caractere intr-un fisier

2. Memorie (text in RAM)

Tip	Clasa	Descriere
Citire	`CharArrayReader`	Citeste caractere dintr-un array de `char`
Scriere	`CharArrayWriter`	Scrie caractere intr-un array de `char`
Citire	`StringReader`	Citeste dintr-un `String`
Scriere	`StringWriter`	Scrie caractere intr-un `String` (intern)

CHARACTER STREAMS

- 3. **Buffered (pentru performanta si metode utile)**

Tip	Clasa	Descriere
Citire	`BufferedReader`	Citeste rapid caractere, linii, etc.
Scriere	`BufferedWriter`	Scrie caractere cu buffer

- 4. **Scriere formatata**

Tip	Clasa	Descriere
Scriere	`PrintWriter`	Scriere formatata de text (cu metode ca `print()`, `println()`, `printf()`), util pentru afisare in fisiere, consola sau retea

endava

CHARACTER STREAMS

🔹 5. Convertire intre byte si caractere

Tip	Clasa	Descriere
Citire	`InputStreamReader`	Converteste un `InputStream` in `Reader`
Scriere	`OutputStreamWriter`	Converteste un `OutputStream` in `Writer`

> ⚠️ Folosite frecvent pentru a transforma byte streams in character streams, mai ales cand lucrezi cu encoding (`UTF-8`, `ISO-8859-1` etc.)

TRY-WITH-RESOURCES

- **Try-with-resources** permite gestionarea automata a resurselor (fisiere, stream-uri, socket-uri etc.) care trebuie inchise dupa utilizare.
- O resursa este orice obiect care implementeaza interfata AutoCloseable (sau Closeable).
  - Exemple: FileInputStream, BufferedReader, Scanner, Socket, Connection

```java
try (ResourceType res = new ResourceType()) {
    // folosesti resursa
} catch (Exception e) {
    // tratare exceptie
}
// resursa se inchide automat aici
```

```java
try (InputStream in = new FileInputStream("fisier.txt")) {
    int data = in.read();
    while (data != -1) {
        System.out.print((char) data);
        data = in.read();
    }
} catch (IOException e) {
    e.printStackTrace();
}
```

NI/O 2 STREAMS

- **NI/O 2 streams** sunt clase mai recente, alternative mai performante la clasele din **java.io**
- Disponibile in Java 7, in pachetul **java.nio.file**
- Clase principale:
  - **Path:** in loc de File; reprezinta o cale catre un fisier sau director
  - **Paths:** metode statice pentru a crea obiecte de tipul Path
  - **Files:** metode statice pentru operatii comune (copiere, stergere, listare)

NI/O 2 STREAMS

- **Paths**

Metoda	Descriere
`Paths.get(String first, String... more)`	Creeaza un obiect `Path` dintr-o cale (relativa sau absoluta).
`Paths.get(URI uri)`	Creeaza un `Path` dintr-un URI (ex: `file:///`).

```
Path p1 = Paths.get("folder", "fisier.txt");
Path p2 = Paths.get("C:/test/fisier.txt");
```

NI/O 2 STREAMS

- **Path**

Metoda	Descriere
`getFileName()`	Returneaza numele fisierului sau ultimului element
`getParent()`	Returneaza directorul parinte
`getRoot()`	Returneaza radacina caii (ex: `C:\`)
`isAbsolute()`	Verifica daca calea este absoluta
`toAbsolutePath()`	Converteste intr-o cale absoluta
`normalize()`	Curata calea (ex: elimina `.` si `..`)
`resolve(Path other)`	Adauga un subpath la un path existent
`relativize(Path other)`	Creeaza un path relativ intre doua cai
`startsWith()` / `endsWith()`	Verifica inceputul sau sfarsitul caii
`toUri()`	Converteste path-ul intr-un URI

NI/O 2 STREAMS

- **Files**

Metoda	Descriere
`exists(path)`	Verifica daca fisierul/directorul exista
`notExists(path)`	Verifica daca nu exista
`createFile(path)`	Creeaza un fisier nou (daca nu exista)
`createDirectory(path)`	Creeaza un director nou
`delete(path)`	Sterge fisierul sau directorul
`deleteIfExists(path)`	Sterge daca exista (fara exceptie daca nu exista)
`copy(source, target, options...)`	Copiaza fisierul dintr-o locatie in alta
`move(source, target, options...)`	Muta sau redenumeste fisierul
`readString(path)`	Citeste continutul unui fisier ca un singur `String`

endava

NI/O 2 STREAMS

- **Files**

`readString(path)`	Citeste continutul unui fisier ca un singur `String`
`writeString(path, text, options...)`	Scrie un `String` intr-un fisier
`readAllLines(path)`	Citeste toate liniile ca `List<String>`
`newBufferedReader(path)`	Creeaza un `BufferedReader` pentru citire
`newBufferedWriter(path)`	Creeaza un `BufferedWriter` pentru scriere
`list(path)`	Returneaza stream cu fisiere/directoare (nu recursiv)
`walk(path)`	Returneaza stream recursiv (toate fisierele/subfolderele)
`isDirectory(path)`	Verifica daca este director
`isRegularFile(path)`	Verifica daca este fisier
`size(path)`	Returneaza dimensiunea fisierului in bytes

Q&A

THANK YOU

DANIELA SPILCĂ